

eXpress Data Paths (XDP)



Mainack Mondal
Sandip Chakraborty

CS 60203
Autumn 2026

Outline

- Why XDP? The cost of a packet in the kernel
- The base idea: a programmable hook inside the NIC driver
- How it works: actions, packet buffer, execution model
- XDP modes and AF_XDP
- XDP vs the Linux stack vs kernel bypass
- Applications
- Case studies: Katran (Meta) and Cilium
- Limitations and summary

Why XDP? The Per-Packet Time Budget

Time per 64-byte packet at line rate

67 ns
10 Gbps: 14.88 million packets/s

16.8 ns
40 Gbps: 59.5 million packets/s

6.7 ns
100 Gbps: 148.8 million packets/s

A single cache miss to DRAM costs around 100 ns.

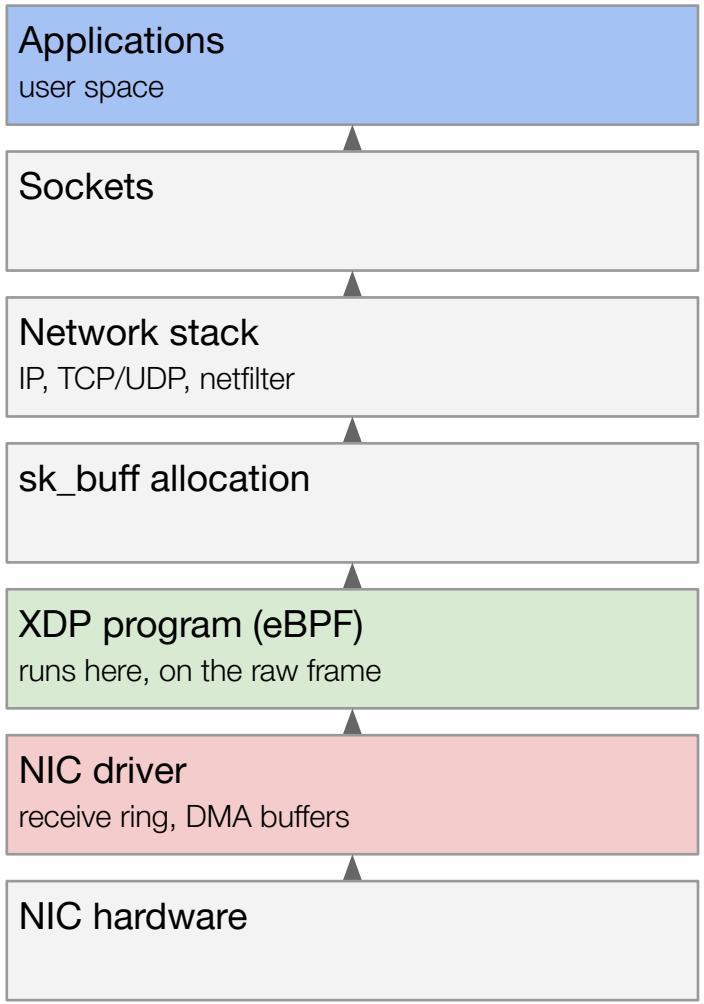
Where the Linux stack spends time per packet

- Allocate and fill an sk_buff (a large metadata structure) for every packet
- Walk through many layers: netfilter, routing, protocol handlers, sockets
- Locks, cache misses, and copies to user space with system calls
- General-purpose code: pays for features a simple filter does not need

The question XDP answers

Kernel bypass (DPDK) reaches line rate, but takes the NIC away from the kernel: no stack, no tools, dedicated busy-polling cores. XDP asks: can we get most of that speed while staying inside the kernel?

The Base Idea of XDP



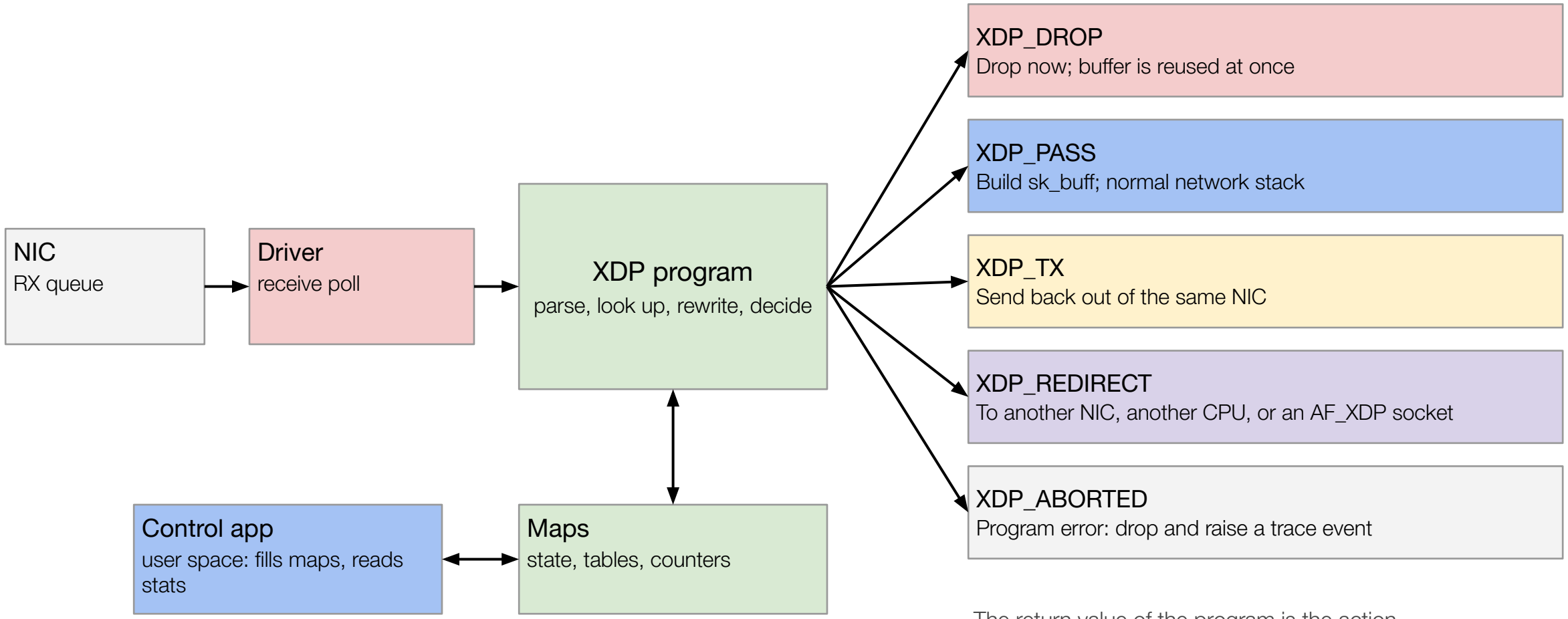
← earliest point in software

packets flow upwards

- Run an **eBPF program for every received packet**, inside the NIC driver, **before** the kernel builds an sk_buff
- The program reads, and can rewrite, the raw frame; then it returns a **verdict**: drop, pass, send back, or redirect
- The verifier keeps it safe; the JIT makes it run at native speed
- **Works with the kernel, not against it**: packets the program does not handle go up the normal stack
- Load or replace the program at run time without stopping traffic
- No dedicated cores, no busy polling: CPU use follows the load

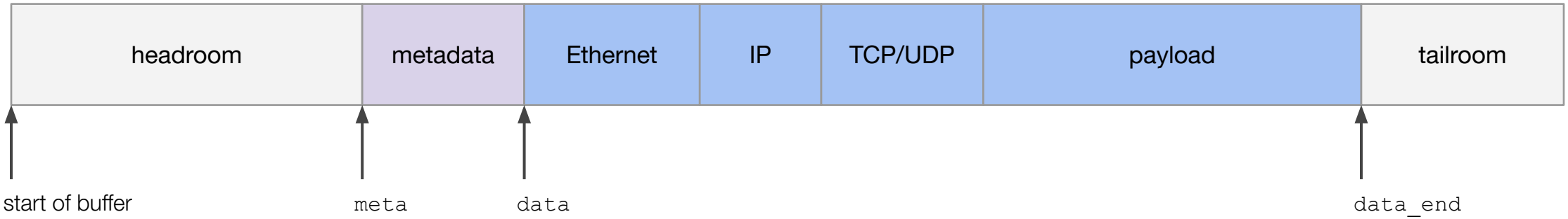
Merged in Linux 4.8 (2016). Høiland-Jørgensen et al., CoNEXT 2018.

XDP Architecture and Actions



The return value of the program is the action.

Inside the Packet Buffer



one memory page (or fixed slot) per frame; headroom is usually 256 B and holds the metadata area

Grow or shrink the packet

- Headroom lets the program add headers in front: encapsulate for a tunnel or a load balancer
- Or remove headers (decapsulate) and trim the tail

Pass hints along

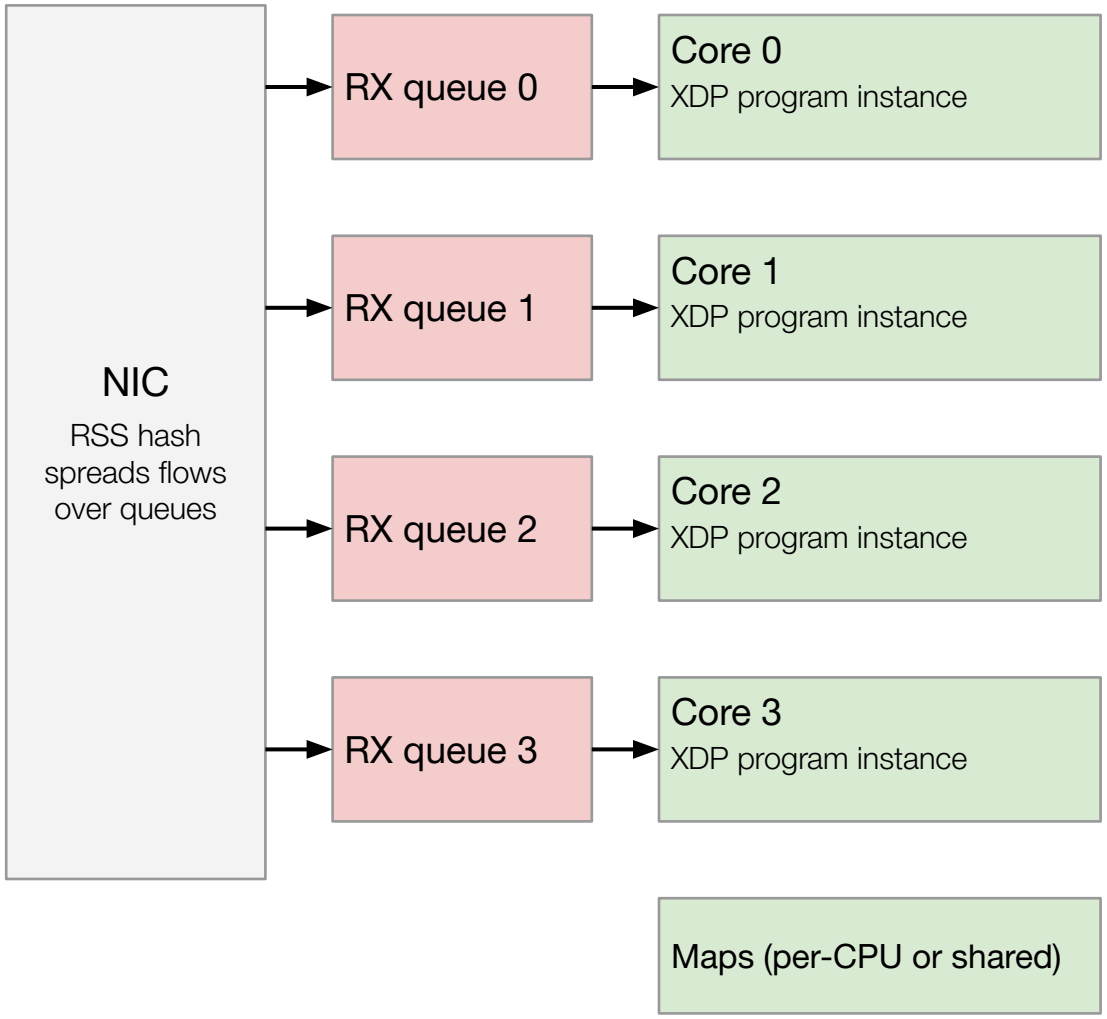
- A small metadata area just before the packet
- Carry a mark or hash to a later tc program or an AF_XDP user

Bounds are checked

- Every access must stay below data_end
- The verifier rejects any read it cannot prove is safe

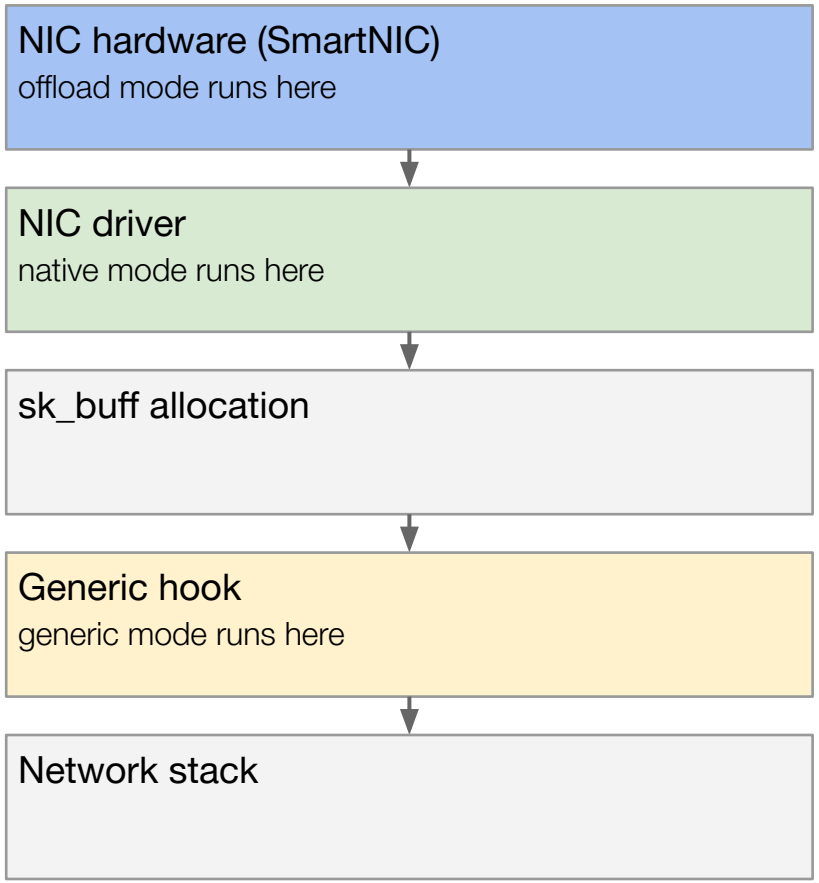
Frames larger than one page (jumbo frames) need multi-buffer support, added in Linux 5.18.

Execution Model: Per Queue, Run to Completion



- **Same program, every queue:** each RX queue is served by one core, so cores do not contend
- **Interrupt, then poll:** the driver polls in batches under load, and sleeps when idle; no wasted CPU
- **Run to completion:** each packet is fully handled by the program; no queues inside XDP
- **Batched redirects:** redirected frames are collected and flushed together at the end of each poll round
- **Per-CPU maps** avoid locks; user space adds the per-CPU values when it reads them

Three Ways to Run XDP



Native (driver) mode

- The normal choice; fastest on the host CPU
- Needs XDP support in the NIC driver (most modern 10G+ drivers)

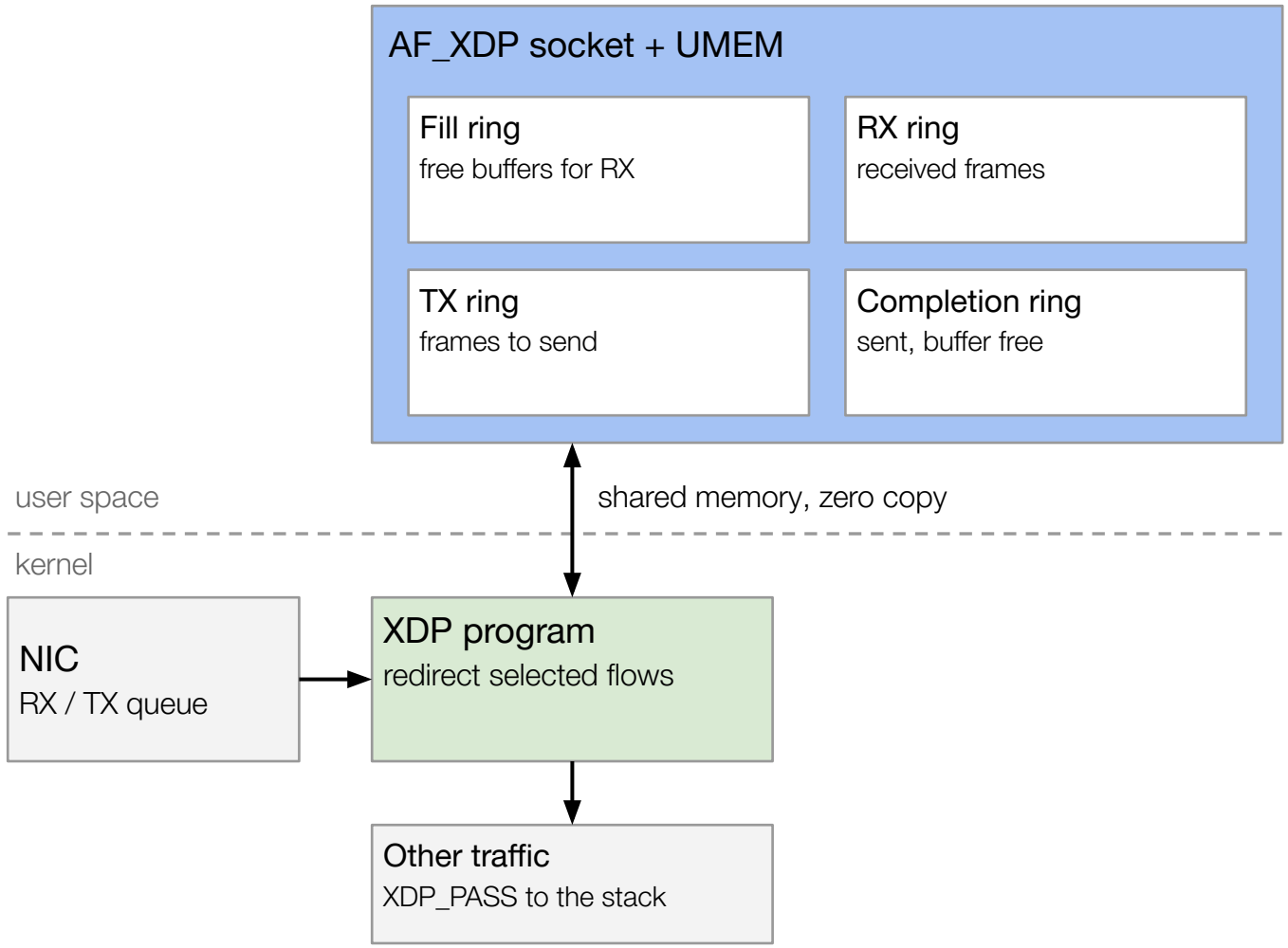
Generic (SKB) mode

- Runs after the sk_buff is built, so it works on any NIC
- Much slower; good for learning and testing

Offload mode

- Runs on the NIC processor itself; host CPU stays free
- Only a few SmartNICs support it, with a smaller feature set

AF_XDP: A Fast Path to User Space



- An XDP program redirects chosen packets into a special socket
- **UMEM:** a memory area shared by the application and the NIC driver
- Four rings pass buffer addresses back and forth; no system call per packet
- **Zero-copy mode** needs driver support; copy mode works everywhere
- Speed close to DPDK, but the **NIC stays under kernel control**: other traffic still uses the normal stack

XDP vs Linux Stack vs Kernel Bypass

Aspect	Linux stack	XDP	Kernel bypass (DPDK)
Where packets are handled	Full kernel stack	NIC driver, inside the kernel	User space, own driver
Packet rate per core	Lowest	High; close to bypass for simple tasks	Highest
CPU use when idle	Near zero	Near zero (interrupt driven)	100% of polling cores
Kernel features and tools	All	Kept: unhandled packets go to the stack	Lost on that NIC
Safety	Trusted kernel code	Verifier-checked program	Process isolation
Changing the logic	Kernel patch or module	Load a new program, no traffic loss	Restart the application
Best for	General traffic	Fast filters, load balancers, forwarding	Complete user-space network functions

XDP is not a full stack. It is a fast, programmable front door in front of the normal stack.

Applications of XDP

DDoS mitigation

- Drop attack packets at line rate, before the stack spends any CPU on them
- Used at the edge by Cloudflare and others

Layer 4 load balancing

- Pick a backend, encapsulate, and send out with XDP_TX
- Katran (Meta), Cilium

Firewalls and ACLs

- Blocklists and prefix rules with hash and LPM trie maps
- Much faster than iptables for simple rules

Forwarding and routing

- Look up the kernel routing table and redirect to the next NIC
- A software router fast path

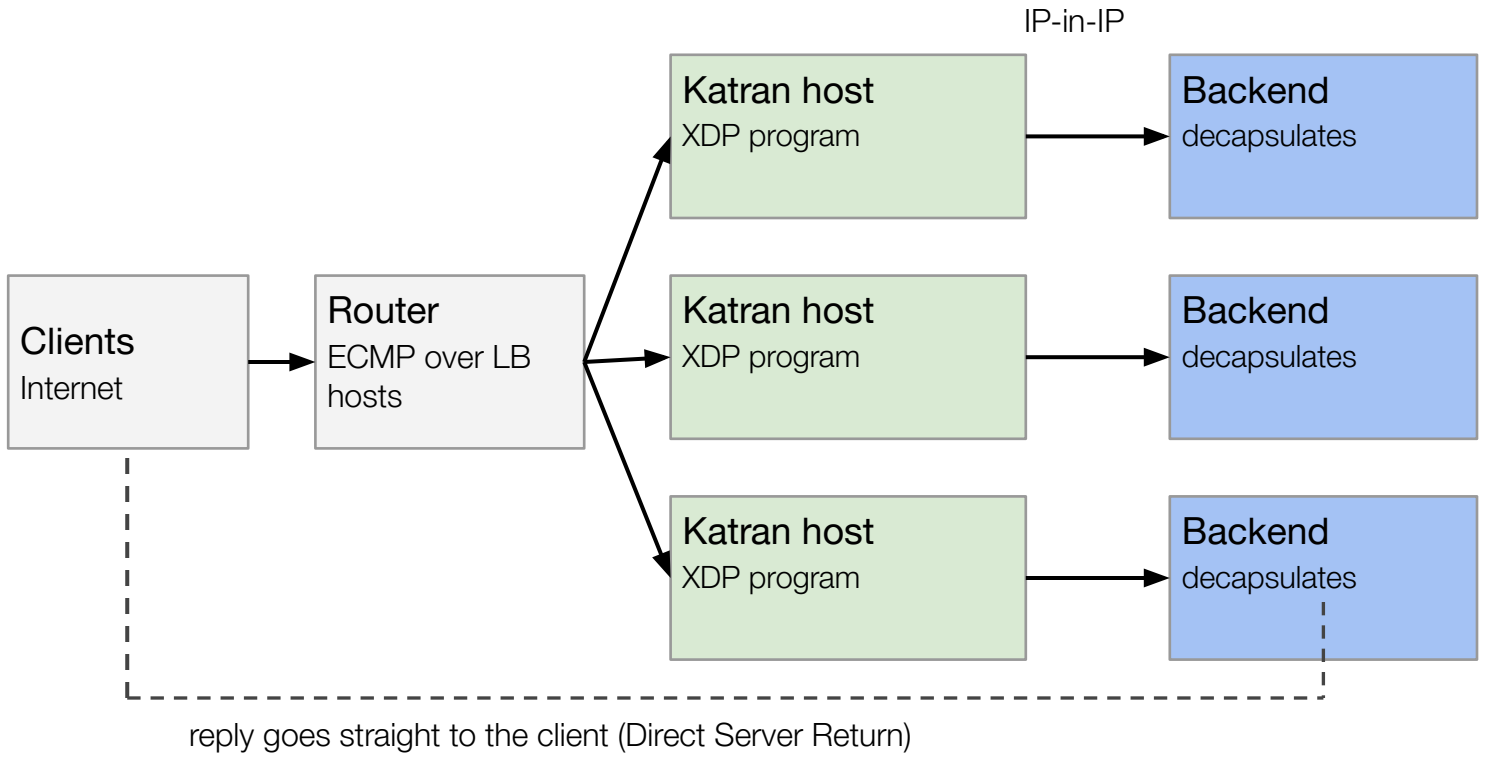
Monitoring and sampling

- Count flows, measure traffic, sample packets
- Almost no effect on forwarding speed

Fast user-space apps

- AF_XDP for packet capture, IDS (e.g. Suricata), custom protocols
- Without giving the NIC away

Case Study: Katran, Meta's Layer 4 Load Balancer



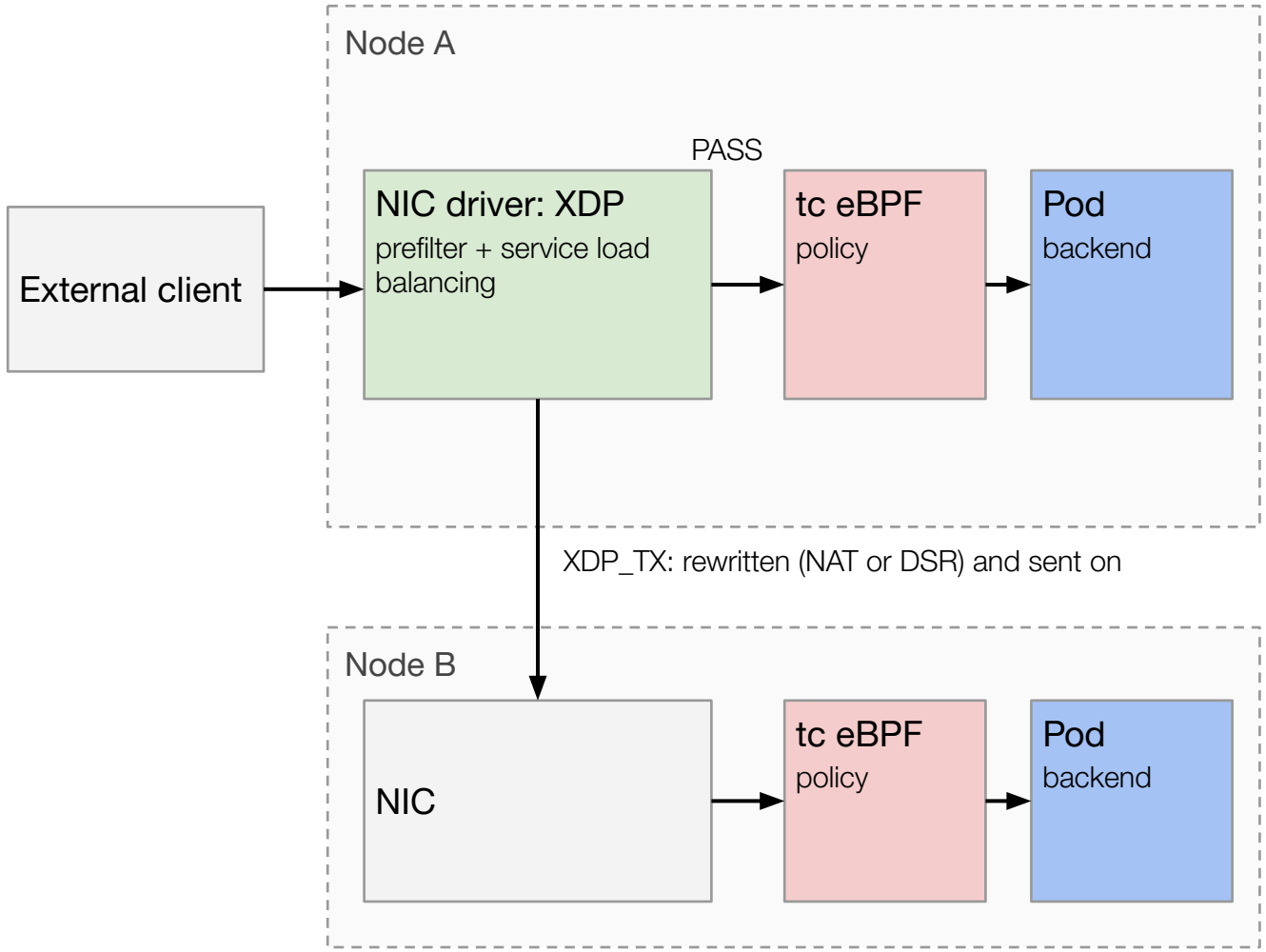
Why XDP fits

Replaced Meta's earlier IPVS-based load balancer; open source since 2018. Scales with the number of RX queues, needs no busy polling, and keeps the normal kernel stack for all other traffic on the host.

Inside the XDP program

- Parse the packet; look up the virtual IP (VIP) in a map
- Known flow? An LRU connection table sends it to the same backend
- New flow? Pick a backend with consistent hashing (Maglev-style), so few flows move when backends change
- Encapsulate in IP-in-IP and send out with XDP_TX
- Non-VIP traffic: XDP_PASS to the normal stack

Case Study: Cilium, eBPF Networking for Kubernetes



How Cilium uses XDP

- **Service load balancing** for NodePort and LoadBalancer services: replaces kube-proxy for traffic from outside
- Backend chosen by Maglev hashing; if it is on another node, the packet is rewritten and bounced out with XDP_TX, never entering Node A's stack
- **Prefilter:** drop traffic from blocked CIDR ranges at the driver

The rest of the datapath

- tc eBPF on pod interfaces: policy and NAT
- Socket-level eBPF: service LB for pod-to-service traffic
- Together they remove iptables from the path

Limitations of XDP

What XDP cannot do easily

- Receive path only; no hook on the transmit side
- No sk_buff yet: no GRO, no socket, little metadata from the NIC
- Not suited for TCP endpoint work (connections, reassembly)
- Complex stateful logic is hard under verifier limits

Practical issues

- Full speed needs native driver support
- One program per interface; several need a dispatcher program (libxdp)
- Jumbo frames need multi-buffer support (5.18+)
- Debugging is harder: packets dropped in XDP are invisible to tcpdump

For full TCP processing at high connection rates, we need a different idea: a fast TCP stack. That is mTCP, our next topic.

Summary

- At 10 to 100 Gbps there are only nanoseconds per packet; the general kernel stack is too heavy for that
- XDP runs an eBPF program in the NIC driver, before the sk_buff, on the raw frame
- Five actions: drop, pass, transmit back, redirect, abort
- Per-queue, run-to-completion, batched; no idle CPU cost; safe through the verifier
- Native, generic and offload modes; AF_XDP gives a zero-copy path to user space
- Used for DDoS defence, firewalls, routing, monitoring and load balancing
- Katran: XDP load balancer with consistent hashing, IP-in-IP and DSR; Cilium: XDP service LB and prefilter for Kubernetes

References

- T. Høiland-Jørgensen et al., The eXpress Data Path: Fast Programmable Packet Processing in the Operating System Kernel, CoNEXT 2018
- Meta Engineering, Open-sourcing Katran, a scalable network load balancer, 2018; github.com/facebookincubator/katran
- Cilium documentation: Kubernetes without kube-proxy, XDP acceleration, standalone load balancer
- D. Eisenbud et al., Maglev: A Fast and Reliable Software Network Load Balancer, NSDI 2016
- Linux kernel documentation: AF_XDP
- XDP tutorial, xdp-project on GitHub